Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 3:
Server TCP con I/O Multiplexing

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement

L'obiettivo della terza esercitazione (Hands-On 3) è il superamento dei limiti dei server iterativi tramite l'implementazione di un'architettura a **I/O Multiplexing**. Il sistema deve essere capace di gestire simultaneamente fino a 10 connessioni client utilizzando un unico thread di esecuzione. La sfida consiste nel monitorare dinamicamente lo stato di molteplici file descriptor e rispondere a ciascun client con un payload specifico (`ACK_MULTIPLEX_OK`), garantendo l'assenza di interferenze tra le sessioni e ottimizzando l'uso del tempo di CPU.

2 Stato dell'Arte

Il multiplexing è la risposta alla necessità di scalabilità senza l'overhead computazionale dei thread o della creazione di processi pesanti (`fork`).

- **Gestione degli Eventi**: Invece di bloccarsi su un singolo socket, il server delega al kernel il monitoraggio di un set di FD. La syscall `select()` analizza tre maschere di bit (lettura, scrittura, eccezione) e ritorna quando almeno un evento è occorso.
- **Limiti della `select()`**: Sebbene portabile, la `select()` presenta un limite strutturale definito dalla costante `FD_SETSIZE` (solitamente 1024). Inoltre, la necessità di scansionare l'intero set ogni volta che la funzione ritorna rende la complessità lineare $O(N)$, dove N è l'ID del descrittore più alto.

3 Metodologia ed Implementazione

L'architettura proposta si basa su un array statico di socket (`client_socket[10]`) e l'uso delle macro POSIX per la manipolazione degli `fd_set`.

3.1 Algoritmo di Multiplexing

Il server esegue un loop continuo in cui ricostruisce la maschera di lettura prima di ogni invocazione della `select()`:

Snippet Gestione Descrittori

```
FD\_ZERO(&readfds);
FD\_SET(master\_socket, &readfds);
for (i = 0; i < MAX\_CLIENTS; i++) {
    sd = client\_socket[i];
    if (sd > 0) FD\_SET(sd, &readfds);
    if (sd > max\_sd) max\_sd = sd;
}
activity = select(max\_sd + 1, &readfds, NULL, NULL, NULL);
```

3.2 Dispatching dei Task

Dopo che la `select()` ritorna, il server distingue due rami logici:

1. **Nuova Connessione**: Se `FD_ISSET(master_socket)`, viene eseguita la `accept()` e il nuovo FD viene memorizzato nel primo slot libero dell'array.

2. **I/O Applicativo**: Se `FD_ISSET(sd)`, viene eseguita la `read()`. Se la lunghezza è 0, il client ha chiuso la socket; altrimenti, il messaggio viene processato.

3.3 Istruzioni di Esecuzione

Compilazione del server multiplexing e del client di test:

Compilation

```
$ gcc server\_multiplex.c -o server
$ gcc client\_test.c -o client
```

4 Risultati

La validazione è stata effettuata tramite uno script bash che lancia 5 client in background, verificando la corretta ricezione degli ACK sincronizzati.

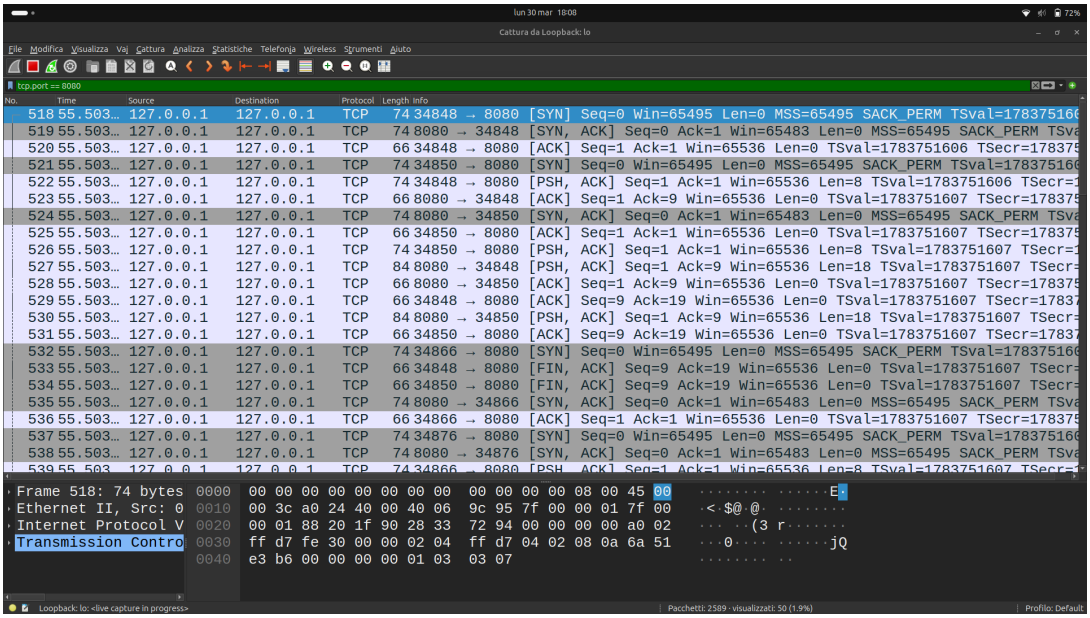

Figure 1: Gestione asincrona del Multiplexing su Wireshark.

4.1 Analisi Wireshark

L'ispezione dei pacchetti (Figura ??multiplexingfig:ho3_multiplexingtra come il server gestisca gli handshake e gli scambi dati in modo asincrono. Nonostante le richieste arrivino in un lasso di tempo di millisecondi, il server risponde a ciascun client (porte sorgente 34848, 34850, ecc.) senza che l'elaborazione di uno blocchi l'altro.

4.2 Osservazione sui Descrittori

L'analisi dei log evidenzia come i file descriptor vengano riutilizzati efficientemente dal kernel una volta chiusi, mantenendo basso l'indice dell'array monitorato dalla `select()`.

5 Conclusioni e Sviluppi Futuri

L'Hands-On 3 ha confermato che il multiplexing è una soluzione robusta per server con numero di client moderato. L'approccio a singolo thread semplifica la gestione della memoria condivisa, eliminando la necessità di primitive di sincronizzazione (mutex).
Sviluppi futuri:

- **Scalabilità**: Migrazione verso `epoll()` (Edge-Triggered) per superare i limiti di `FD_SETSIZE` in scenari di alto traffico.
- **Persistenza**: Implementazione di un protocollo applicativo con gestione del mantenimento dello stato (session tracking) tra invii consecutivi dello stesso client.